

Design Concepts

1. Coupling

1.1. Content coupling

Related modules	Description	Improvement Direction

1.2. Common coupling

Related modules	Description	Improvement Direction

1.3. Control coupling

Related modules	Description	Improvement Direction
PaymentController → PayByCreditCardService	The controller determines the payment type (credit card) and delegates processing to the service layer.	Use polymorphism (Strategy Pattern) to remove hard-coded control logic.
PayByCreditCardService → OrderRepository	The service controls when and how order data is retrieved and updated during payment processing.	Encapsulate business rules in domain/service layer to reduce direct control flow.
PayByCreditCardService → PaymentTransactionRepository	The service explicitly controls transaction creation and persistence.	Introduce TransactionService to decouple responsibilities.

1.4. Stamp coupling

Related modules	Description	Improvement Direction
PlaceOrderService → Order, OrderDraftContext, OrderMapper	PlaceOrderService exchanges complete Order aggregate objects with OrderDraftContext and OrderMapper during order drafting, invoice generation, order finalization, and response mapping. The service depends on composite domain objects rather than only primitive values because order-related operations	Current design is acceptable because Order acts as the aggregate root of the ordering process. Further reduction of coupling may unnecessarily complicate the design.

	require access to multiple order attributes and associated entities.	
DeliveryService → OrderDraftContext, OrderMapper	DeliveryService depends on complete Order and DeliveryInformation objects retrieved through OrderDraftContext and passed to OrderMapper. The service accesses multiple attributes related to delivery processing, delivery fee calculation, and response generation.	Coupling level is acceptable due to aggregate-oriented business logic. Could introduce specialized DTOs or value objects if delivery logic becomes significantly more complex.
StockValidator → Cart, CartItem, Product, CartContext	StockValidator operates on complete Cart, CartItem, and Product objects to validate stock availability and synchronize cart items with managed product entities. Validation logic requires access to multiple fields such as product identifiers, quantities, stock quantities, and cart operations.	Coupling is acceptable because stock validation naturally depends on aggregate cart and product information. Could separate cart synchronization responsibilities into another component if validation logic grows larger.
OrderDraftContext, SessionOrderDraftProvider → Order	OrderDraftContext and SessionOrderDraftProvider exchange complete Order aggregate objects during draft order persistence and retrieval operations. Full aggregate objects are required because the draft order contains interconnected ordering state and associated entities.	Current design is acceptable because Order acts as the aggregate root of the ordering workflow. Could introduce lightweight draft state objects if session storage becomes excessively large or distributed.
NotificationService & EmailService → OrderSuccessEvent(Order, PaymentTransaction), JavaMailSender	NotificationService and EmailService depend on complete Order and PaymentTransaction objects to construct order confirmation notifications. EmailService additionally depends on JavaMailSender infrastructure for MIME message creation and email transmission.	Current design is acceptable for project scope. Could introduce dedicated notification DTOs or email template models to reduce dependency on domain entities in larger systems.
DeliveryInformation, Invoice → Order	DeliveryInformation and Invoice maintain bidirectional entity associations with Order to preserve aggregate consistency and ORM relationship management. Both entities depend on the complete Order	Bidirectional coupling is common and acceptable in JPA aggregate modeling. Could reduce coupling using unidirectional

	aggregate during creation and lifecycle synchronization.	relationships if navigation from child entities to Order becomes unnecessary.
ProductManagementService → ProductManagementController	The Controller passes entire DTO objects (CreateProductRequest, UpdateProductRequest, DeleteProductRequest) to the Service and receives ProductDetail objects in return.	Current design is acceptable because passing individual fields (strict Data Coupling) for massive DTOs to achieve Data Coupling would result in bloated method signatures and poor maintainability.
ProductManagementService. → ProductFactory	The Service sends DTOs and the original Product entities to the Factory for branching and instantiation. The Factory only extracts the specific attributes it needs to execute the Builders.	Coupling level is acceptable as the Factory requires "raw materials" to construct objects.
ProductManagementService → ProductRepository	The Service passes the entire composite Product entity to the Repository for persistence	Coupling level is acceptable because by passing the entire object, its semantic meaning and the encapsulation of its data is preserved.
PaymentController → PayByCreditCardService (PayByCreditCardRequest)	The entire request object is passed, although only some fields (cardNumber, cvv, amount) are used.	Split DTO into smaller objects such as CardInfo and PaymentInfo.
PayByCreditCardService → Order	The full Order object is used even though only status and ID are required.	Use lightweight DTO or projection instead of full entity.
VietQRController → Order	VietQRController receives the full Order object in generateQRCode() and checkPaymentStatus(), but only uses a small part of the object, mainly order ID and total amount. Therefore, the whole composite object is passed even though only some data is needed.	Replace the full Order parameter with only the required values, such as orderId and amount, or create a smaller payment request DTO.

1.5. Data coupling

Related modules	Description	Improvement Direction
-----------------	-------------	-----------------------

OrderController → PlaceOrderService, DeliveryService, DeliveryRequest (DTO)	OrderController depends on 2 service classes PlaceOrderService and DeliveryService to orchestrate the place order process, including stock validation, order drafting, delivery submission and update, invoice generation, finalization, retrieval, and cancellation. Controller methods send data structures that the 2 service classes use wholly or sends only necessary data to the service classes.	None.
PlaceOrderService → StockValidator, OrderRepository, ApplicationEventPublisher	PlaceOrderService depends on StockValidator for stock checking, OrderRepository for order persistence, and ApplicationEventPublisher for publishing order success events. Interactions are performed through method calls with only the required parameters or return values being exchanged, without exposing internal implementation details.	None.
DeliveryService → DeliveryFeeCalculator	DeliveryService depends on DeliveryFeeCalculator to compute delivery fees based on total weight, province, and order price. Only the necessary primitive/simple values are passed to the calculator instead of the whole Order object, reducing unnecessary dependency between modules.	None.
PlaceOrderService, DeliveryService → OrderDraftContext	PlaceOrderService and DeliveryService depend on the abstraction OrderDraftContext to manage draft order lifecycle operations such as retrieval, storage, and removal. Services interact only through interface methods without depending directly on HTTP session APIs or concrete persistence mechanisms.	None.
SessionOrderDraftProvider → HttpSession	SessionOrderDraftProvider depends on HttpSession to persist draft order data across the ordering workflow. Interaction occurs through session API method calls for attribute retrieval, storage, and removal.	None.
StockValidator → ProductRepository	StockValidator depends on ProductRepository to retrieve active products for stock validation. The repository interaction is performed through dedicated query methods and only the retrieved product data needed for validation is used.	None.
PlaceOrderSuccessListener → NotificationService	PlaceOrderSuccessListener depends on the abstraction NotificationService to send order confirmation notifications after successful order placement. The listener interacts only through the service interface method, reducing direct dependency on concrete notification implementations such as EmailService.	None.

ProductManagementService → ProductManagementController	The Controller calls Service methods using primitive parameters (e.g., passing UUID id in the updateProduct method).	None.
ProductManagementService → ProductRepository	The Service passes primitive identifiers (UUID id) to the Repository for data querying	None.
PayByCreditCardService → OrderRepository	Only necessary data (order ID, status) is passed between service and repository.	No improvement needed — already optimal.
PayByCreditCardService → PaymentTransactionRepository	Only required transaction data (amount, timestamp, status) is passed.	No improvement needed — already optimal.
VietQRController → VietQRBoundary	VietQRController passes only simple String values such as authorizationHeader, accessToken, and requestString to VietQRBoundary and receives a raw JSON String. Only the exact data needed for HTTP communication is exchanged.	None.
VietQRController → QRAccessTokenRequest	VietQRController creates QRAccessTokenRequest using username and password, then calls buildAuthorizationHeader() to receive a String authorization header. Only necessary data is exchanged.	None.
VietQRController → QRAccessTokenResponse	VietQRController passes the raw response String to QRAccessTokenResponse, which parses and stores accessToken, tokenType, and expiresIn. The controller only retrieves the parsed token data through getter methods.	None.
VietQRController → QRGenerateRequest	VietQRController constructs QRGenerateRequest using simple values such as bankCode, accountNo, accountName, content, amount, and orderId. It then calls buildRequestString() to get a request body String.	None.
VietQRController → QRTestCallbackRequest	VietQRController constructs QRTestCallbackRequest using bankAccount, content, amount, and bankCode, then receives a request body String from buildRequestString().	None.
PayOrderService → IPaymentQRCode	PayOrderService communicates with the VietQR subsystem through the IPaymentQRCode interface. It passes an Order object and receives QRCode or QRCodePaymentStatus objects without depending on the concrete VietQR implementation.	None.
PayOrderController → PayOrderService, PlaceOrderService	PayOrderController calls service methods and exchanges required data objects such as Order, QRCode, QRCodePaymentStatus,	None.

	PaymentTransaction, and response DTOs. It does not depend on the internal implementation of the services.	
Product entity classes	The entities are completely self-contained. They do not depend on any services, repositories, controllers, or external infrastructure frameworks.	None

2. Cohesion

2.1. *Coincidental cohesion*

Related modules	Description	Improvement Direction

2.2. *Logical cohesion*

Related modules	Description	Improvement Direction

2.3. *Temporal cohesion*

Related modules	Description	Improvement Direction

2.4. *Procedural cohesion*

Related modules	Description	Improvement Direction
OrderController	Coordinates sequential order placement workflow through REST endpoints. Methods participate in the same order processing procedure: placing order, submitting delivery info, retrieving invoice, retrieving order details, and canceling order. (not concerning payment)	Not entirely necessary. If the requirement grew, the class could be separated into 2 modules: - PlaceOrderController: to control REST endpoints for draft order during the place order session. - CustomerOrderManagementController: to control REST endpoints for finalized placed orders operations.
PlaceOrderService	Coordinates the order placement workflow including stock validation, order drafting,	If the requirement grew, this service class could risk becoming a “God service” class, worsening cohesion. An improvement direction is to move order

	invoice generation, finalization, retrieval, and cancellation.	management methods (get order by ID, cancel finalized order) to a different service class namely CustomerOrderManagementService.
PayByCreditCardService	The service executes a sequence of steps: validate card → retrieve order → simulate payment → create transaction → update order → send notification.	Split into smaller services (CardValidationService, PaymentGatewayService, TransactionService) if system scales.

2.5. Communicational cohesion

Related modules	Description	Improvement Direction
DeliveryService	DeliveryService groups operations related to delivery processing, including creation and update of delivery information, delivery fee recalculation, draft order synchronization, and response mapping. All methods operate on the same delivery-related data and draft order state.	Current cohesion level is acceptable for project scope. If delivery logic grows significantly, responsibilities such as fee calculation, delivery information management, and validation could be separated into dedicated services.
PlaceOrderSuccessListener	PlaceOrderSuccessListener processes post-order-success operations including sending notifications, clearing draft order state, and clearing cart contents. All operations are triggered by and operate on the same OrderSuccessEvent data.	Could separate notification handling and cleanup responsibilities into multiple listeners if the number of post-order actions increases in the future.
EmailService	EmailService coordinates email content construction, MIME message creation, exception handling, and email transmission. All methods collaborate to produce and send order confirmation emails using related email communication data.	Could separate email template generation from email transport responsibilities to improve cohesion if email functionality becomes more complex.
DeliveryInformation	DeliveryInformation encapsulates customer delivery details and maintains association synchronization with the related Order. All attributes and methods operate on the same delivery-related entity data.	Current cohesion is acceptable. Could reduce mutability by replacing public setters with controlled update methods to strengthen encapsulation.
Order	Order acts as the aggregate root for order-related business data,	As the system grows, some responsibilities such as pricing

	including items, delivery information, invoice, payment transaction, status transitions, and pricing calculations. All methods and attributes operate on the same order aggregate state and related business rules.	calculation or invoice generation could be extracted into dedicated domain services to reduce aggregate complexity.
ProductManagementController	All REST API endpoints (createProduct, updateProduct, deleteProducts) inside this controller operate on the exact same resource category: the Product	Further improvement is not currently compulsory as the controller successfully groups all HTTP routing logic related to product management.
ProductManagementService	The internal methods coordinate the lifecycle of a single core data structure (the Product entity). The Service acts as a clean delegator, orchestrating the workflow between the Factory, Mapper, and Repository without implementing the creation logic itself.	Current cohesion level is acceptable because the service is now keeping Create, Update, and Delete (CRUD) operations together, making the code centralized, easy to navigate.

2.6. Sequential cohesion

Related modules	Description	Improvement Direction

2.7. Functional cohesion

Related modules	Description	Improvement Direction
OrderDraftContext	Defines a single abstraction for managing draft order lifecycle operations including retrieval, storage, and clearing of draft orders. All methods contribute solely to draft order context management.	None.
SessionOrderDraftProvider	Manages persistence of draft orders within HTTP session storage. All methods contribute directly to session-based draft order management and retrieval.	None.
DeliveryFeeCalculator	Calculates delivery fee based on Weight, Province, Order Value. All attributes, constants, and methods contribute solely to delivery fee calculation logic.	None.
StockValidator	Validates stock availability and cart consistency before order placement. All methods and logic	None.

	contribute directly to checking product existence, stock quantity, and synchronization of cart items with managed product entities.	
OrderSuccessEvent	Encapsulates data related to a successful order event, including the Order and PaymentTransaction. All fields exist solely to represent event data transferred between publisher and listener modules.	None.
NotificationService	Defines a single abstraction for sending order confirmation notifications. All methods contribute solely to notification service definition and communication contract specification.	None.
Invoice	Represents invoice data and invoice creation logic for an Order. All attributes and methods contribute directly to invoice representation and financial snapshot creation.	None.
OrderItem	Represents a purchased product item within an Order, including quantity, pricing, weight, and related calculations. All methods and fields contribute solely to order item representation and computation.	None.
OrderItemKey	Represents the composite identifier of OrderItem. All fields and methods contribute solely to identity representation and equality/hash comparison logic.	None.
Product entity classes ¹	Entities act as highly encapsulated Abstract Data Types (ADTs). All attributes are tightly protected, and the nested Builder classes focus entirely on a single, well-defined task: validating and constructing a strictly valid domain state.	None
ProductFactory	All of its methods (buildNewProduct, buildUpdatedProduct, buildDeletedProduct) strictly focus on a single responsibility: instantiating and cloning Product subclasses using the nested Builder Pattern.	None.
DTO classes ²	Data structures used to transfer request and response data across the web/API boundary. DTO classes primarily encapsulate related data fields without containing complex business logic.	None.
Repository interfaces ³	JPA persistence interfaces responsible for managing entity retrieval and persistence operations. Repository methods focus solely on	None.

¹ Coupling and cohesion levels of Product entity classes are documented/commented on the package level in the package-info.java file of package com.hust.soict.ict.aims.models.entities.product

² Coupling and cohesion levels of DTO classes are documented/commented on the package level in the package-info.java file of package com.hust.soict.ict.aims.models.dto

³ Coupling and cohesion levels of Repository interfaces are documented/commented on the package level in the package-info.java file of package com.hust.soict.ict.aims.repositories

	data access abstraction and database interaction.	
DTO mappers ⁴	Mapper interfaces responsible for transforming entities into DTOs and vice versa. All mapping methods contribute solely to object transformation between architectural layers.	None.
PayByCreditCardService	All operations contribute to a single responsibility: processing credit card payment.	No improvement needed — high cohesion.
VietQRController	Handles the VietQR payment integration process, including generating access tokens, generating QR codes, checking payment status, and creating test callback requests. All methods support the single purpose of VietQR payment processing.	None.
VietQRBoundary	Handles HTTP communication with VietQR API endpoints. All methods focus on sending requests and receiving responses from the external VietQR service.	None.
QRAccessTokenRequest	Builds the authorization header needed to request an access token from VietQR. All fields and methods support token request preparation.	None.
QRAccessTokenResponse	Represents and parses the access token response from VietQR. All fields and methods support storing and validating token information.	None.
QRGenerateRequest	Builds the QR generation request body. All fields and methods contribute to preparing the request data for QR code generation.	None.
QRTestCallbackRequest	Builds the test callback request body for simulating payment in the VietQR sandbox environment. All fields and methods support this single purpose.	None.
QRCode	Represents and parses QR code information returned from VietQR, including qrCode, qrLink, bankCode, bankName, and bankAccount.	None.
QRCodePaymentStatus	Represents and interprets payment status returned from VietQR. Its methods check whether the payment is completed, pending, failed, or cancelled.	None.
PayOrderService	Coordinates the payment use case by generating QR codes, checking payment status, and creating PaymentTransaction after successful payment.	None.

⁴ Coupling and cohesion levels of DTO mappers are documented/commented on the package level in the package-info.java file of package com.hust.soict.ict.aims.mapper